

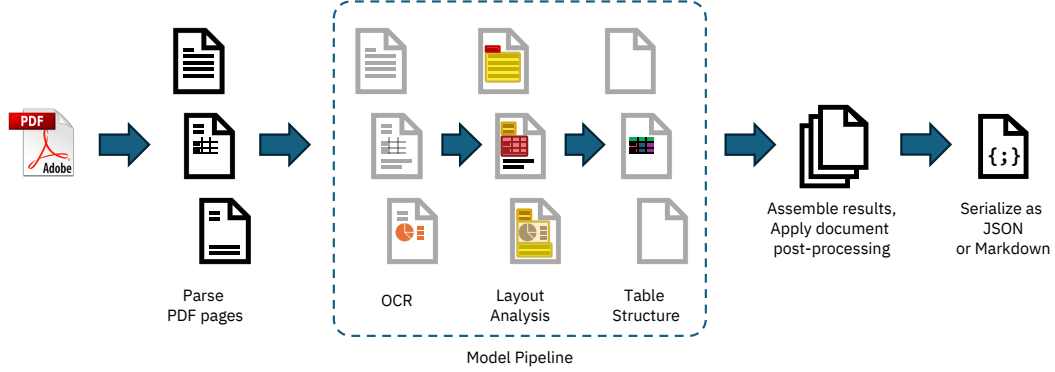


Figure 1: Sketch of Docling’s default processing pipeline. The inner part of the model pipeline is easily customizable and extensible.

licensing (e.g. `pymupdf` [7]), poor speed or unrecoverable quality issues, such as merged text cells across far-apart text tokens or table columns (`pypdfium`, `PyPDF`) [15, 14].

We therefore decided to provide multiple backend choices, and additionally open-source a custom-built PDF parser, which is based on the low-level `qpdf`[4] library. It is made available in a separate package named `docling-parse` and powers the default PDF backend in Docling. As an alternative, we provide a PDF backend relying on `pypdfium`, which may be a safe backup choice in certain cases, e.g. if issues are seen with particular font encodings.

3.2 AI models

As part of Docling, we initially release two highly capable AI models to the open-source community, which have been developed and published recently by our team. The first model is a layout analysis model, an accurate object-detector for page elements [13]. The second model is TableFormer [12, 9], a state-of-the-art table structure recognition model. We provide the pre-trained weights (hosted on [huggingface](#)) and a separate package for the inference code as `docling-ibm-models`. Both models are also powering the open-access `deepsearch-experience`, our cloud-native service for knowledge exploration tasks.

Layout Analysis Model

Our layout analysis model is an object-detector which predicts the bounding-boxes and classes of various elements on the image of a given page. Its architecture is derived from RT-DETR [16] and re-trained on DocLayNet [13], our popular human-annotated dataset for document-layout analysis, among other proprietary datasets. For inference, our implementation relies on the `onnxruntime` [5].

The Docling pipeline feeds page images at 72 dpi resolution, which can be processed on a single CPU with sub-second latency. All predicted bounding-box proposals for document elements are post-processed to remove overlapping proposals based on confidence and size, and then intersected with the text tokens in the PDF to group them into meaningful and complete units such as paragraphs, section titles, list items, captions, figures or tables.

Table Structure Recognition

The TableFormer model [12], first published in 2022 and since refined with a custom structure token language [9], is a vision-transformer model for table structure recovery. It can predict the logical row and column structure of a given table based on an input image, and determine which table cells belong to column headers, row headers or the table body. Compared to earlier approaches, TableFormer handles many characteristics of tables, such as partial or no borderlines, empty cells, rows or columns, cell spans and hierarchy both on column-heading or row-heading level, tables with inconsistent indentation or alignment and other complexities. For inference, our implementation relies on `PyTorch` [2].

The Docling pipeline feeds all table objects detected in the layout analysis to the TableFormer model, by providing an image-crop of the table and the included text cells. TableFormer structure predictions are matched back to the PDF cells in post-processing to avoid expensive re-transcription text in the table image. Typical tables require between 2 and 6 seconds to be processed on a standard CPU, strongly depending on the amount of included table cells.

OCR

Docling provides optional support for OCR, for example to cover scanned PDFs or content in bitmaps images embedded on a page. In our initial release, we rely on *EasyOCR* [1], a popular third-party OCR library with support for many languages. Docling, by default, feeds a high-resolution page image (216 dpi) to the OCR engine, to allow capturing small print detail in decent quality. While EasyOCR delivers reasonable transcription quality, we observe that it runs fairly slow on CPU (upwards of 30 seconds per page).

We are actively seeking collaboration from the open-source community to extend Docling with additional OCR backends and speed improvements.

3.3 Assembly

In the final pipeline stage, Docling assembles all prediction results produced on each page into a well-defined datatype that encapsulates a converted document, as defined in the auxiliary package *docling-core*. The generated document object is passed through a post-processing model which leverages several algorithms to augment features, such as detection of the document language, correcting the reading order, matching figures with captions and labelling metadata such as title, authors and references. The final output can then be serialized to JSON or transformed into a Markdown representation at the users request.

3.4 Extensibility

Docling provides a straight-forward interface to extend its capabilities, namely the model pipeline. A model pipeline constitutes the central part in the processing, following initial document parsing and preceding output assembly, and can be fully customized by sub-classing from an abstract base-class (*BaseModelPipeline*) or cloning the default model pipeline. This effectively allows to fully customize the chain of models, add or replace models, and introduce additional pipeline configuration parameters. To use a custom model pipeline, the custom pipeline class to instantiate can be provided as an argument to the main document conversion methods. We invite everyone in the community to propose additional or alternative models and improvements.

Implementations of model classes must satisfy the python `Callable` interface. The `__call__` method must accept an iterator over page objects, and produce another iterator over the page objects which were augmented with the additional features predicted by the model, by extending the provided `PagePredictions` data model accordingly.

4 Performance

In this section, we establish some reference numbers for the processing speed of Docling and the resource budget it requires. All tests in this section are run with default options on our standard test set distributed with Docling, which consists of three papers from arXiv and two IBM Redbooks, with a total of 225 pages. Measurements were taken using both available PDF backends on two different hardware systems: one MacBook Pro M3 Max, and one bare-metal server running Ubuntu 20.04 LTS on an Intel Xeon E5-2690 CPU. For reproducibility, we fixed the thread budget (through setting *OMP_NUM_THREADS environment variable*) once to 4 (Docling default) and once to 16 (equal to full core count on the test hardware). All results are shown in Table 1.

If you need to run Docling in very low-resource environments, please consider configuring the *pypdfium* backend. While it is faster and more memory efficient than the default *docling-parse* backend, it will come at the expense of worse quality results, especially in table structure recovery.

Establishing GPU acceleration support for the AI models is currently work-in-progress and largely untested, but may work implicitly when CUDA is available and discovered by the *onnxruntime* and

torch runtimes backing the Docling pipeline. We will deliver updates on this topic at in a future version of this report.

Table 1: Runtime characteristics of Docling with the standard model pipeline and settings, on our test dataset of 225 pages, on two different systems. OCR is disabled. We show the time-to-solution (TTS), computed throughput in pages per second, and the peak memory used (resident set size) for both the Docling-native PDF backend and for the pypdfium backend, using 4 and 16 threads.

CPU	Thread budget	native backend			pypdfium backend		
		TTS	Pages/s	Mem	TTS	Pages/s	Mem
Apple M3 Max (16 cores)	4	177 s	1.27	6.20 GB	103 s	2.18	2.56 GB
	16	167 s	1.34		92 s	2.45	
Intel(R) Xeon E5-2690 (16 cores)	4	375 s	0.60	6.16 GB	239 s	0.94	2.42 GB
	16	244 s	0.92		143 s	1.57	

5 Applications

Thanks to the high-quality, richly structured document conversion achieved by Docling, its output qualifies for numerous downstream applications. For example, Docling can provide a base for detailed enterprise document search, passage retrieval or classification use-cases, or support knowledge extraction pipelines, allowing specific treatment of different structures in the document, such as tables, figures, section structure or references. For popular generative AI application patterns, such as retrieval-augmented generation (RAG), we provide *quackling*, an open-source package which capitalizes on Docling’s feature-rich document output to enable document-native optimized vector embedding and chunking. It plugs in seamlessly with LLM frameworks such as LlamaIndex [8]. Since Docling is fast, stable and cheap to run, it also makes for an excellent choice to build document-derived datasets. With its powerful table structure recognition, it provides significant benefit to automated knowledge-base construction [11, 10]. Docling is also integrated within the open IBM data prep kit [6], which implements scalable data transforms to build large-scale multi-modal training datasets.

6 Future work and contributions

Docling is designed to allow easy extension of the model library and pipelines. In the future, we plan to extend Docling with several more models, such as a figure-classifier model, an equation-recognition model, a code-recognition model and more. This will help improve the quality of conversion for specific types of content, as well as augment extracted document metadata with additional information. Further investment into testing and optimizing GPU acceleration as well as improving the Docling-native PDF backend are on our roadmap, too.

We encourage everyone to propose or implement additional features and models, and will gladly take your inputs and contributions under review. The codebase of Docling is open for use and contribution, under the MIT license agreement and in alignment with our contributing guidelines included in the Docling repository. If you use Docling in your projects, please consider citing this technical report.

References

- [1] J. AI. Easyocr: Ready-to-use ocr with 80+ supported languages. <https://github.com/JaidedAI/EasyOCR>, 2024. Version: 1.7.0.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala. Pytorch 2: Faster